

VPS DEPLOYMENT AND UPDATE GUIDE

GGWP Boost

Production step-by-step guide for updating the existing Laravel 12 + Blade + Bootstrap site on a VPS from a freshly cloned or newly pulled repository copy.

PROJECT TYPE

Laravel 12 application with Blade, Bootstrap, Vite, database-backed queues/cache/sessions, realtime chat, Stripe, and Cryptomus.

GUIDE GOAL

Safely deploy the latest code to a VPS, run migrations, build assets, refresh caches, restart workers/services, and verify the site after the update.

GENERATED FROM

Actual repository inspection of the GGWP Boost codebase.

Generated for VPS deployment/update use. Adjust server-specific paths, usernames, service names, and repo URLs to match your environment.

1. Overview

This guide is for updating an already-running GGWP Boost site on a VPS. The project is not a simple static Laravel app. It includes:

- Laravel 12 on PHP 8.2+
- Blade views with Bootstrap and Vite-built frontend assets
- database-backed sessions, cache, queue jobs, and failed jobs by default
- Stripe and Cryptomus payment flows with webhook handling
- Pusher-compatible realtime chat using `digiworld/laravel-websockets`
- queued customer emails and Discord notifications
- a scheduler that prunes stale pending checkouts and retries Discord dispatches

The update process in this guide clones or refreshes the latest code, prepares a new release, runs the production build and migrations safely, switches live traffic to the new release, restarts the long-running processes, and verifies the application with codebase-aware checks.

2. Assumptions

OS

Ubuntu or Debian style Linux VPS. Commands are written for a Bash shell. Adjust package manager and service names if your VPS differs.

Web stack

Nginx + PHP-FPM is assumed because that is the most common production layout for a Laravel VPS. Apache can work too, but you will need to adjust the web-server restart/reload commands.

Deployment user

A non-root deploy user such as `deploy`, with the web-server group set to `www-data`.

Project layout

A release-based layout is recommended:
`/var/www/ggwp-boost/releases/<timestamp>`
`/var/www/ggwp-boost/shared`
`/var/www/ggwp-boost/current -> active release`

Database

MySQL is assumed for production because the repository defaults to MySQL in `.env.example` and expects database-backed queue/cache/session state.

If your current VPS uses a single fixed app directory instead of a `current` symlink with releases, you can still follow this guide, but adapt the switch step accordingly. The release-based layout is strongly

recommended because this codebase has queued jobs, asset builds, and migrations that are safer to roll forward and roll back when releases are separated.

3. Requirements / Prerequisites

Server packages and binaries

- `git`
- `php 8.2+`
- `composer`
- `node` and `npm` for the Vite production build
- `mysql-client` or equivalent if you want shell-level backups with `mysqldump`
- `nginx` or `apache2`
- `php-fpm` if you use Nginx
- a process manager for the queue worker and websocket server:
 - `systemd` preferred, or
 - `supervisor`

Likely PHP extensions

The project itself confirms or strongly implies these are needed:

- `pdo_mysql` or the matching PDO extension for your production database
- `mbstring`
- `openssl`
- `tokenizer`
- `xml`
- `ctype`
- `json`
- `fileinfo`
- `gd` for image upload sanitization and re-encoding
- `exif` recommended because JPEG orientation correction is implemented

Access and config prerequisites

- working production `.env` values
- database credentials with permission to run migrations
- mail credentials
- payment provider credentials and webhook secrets
- broadcast/websocket host and key settings that match how browsers reach the websocket endpoint

4. Project Stack Summary

Application stack

- Laravel ^12.0
- PHP ^8.2
- Blade-first frontend
- Bootstrap 5.3.3 from CDN
- Vite 7 build pipeline
- Tailwind plugin present for the CSS build pipeline

Operational services

- database-backed queue by default
- database-backed cache by default
- database-backed sessions by default
- Laravel WebSockets server for chat
- cron-driven scheduler
- customer email and Discord notification queues

Payments

Stripe Checkout and Cryptomus hosted invoices. Both have webhook endpoints and both depend on correct `.env` credentials. The app also performs success-page verification and stores durable pending checkouts before redirecting customers off-site.

Realtime chat

Private broadcast channels for order chat, powered by a self-hosted Pusher-compatible websocket server. In production, this means the websocket process must stay up and the browser must be able to reach the configured websocket host/port/scheme.

Scheduler

Runs `pending-checkouts:prune` daily and `discord:retry-dispatches` every ten minutes.

Readiness endpoint

`/ready` checks database, cache, default filesystem write access, and the `public/storage` symlink. This is one of your best post-deploy verification points for this project.

Startup validation

The app validates critical config on boot, and in `production` / `staging` it becomes stricter about integrations. Broken payment or runtime config can stop the app from booting cleanly, so stale `.env` or bad cache is a real deploy risk here.

5. Safe Deployment Strategy

1. Back up the current `.env`.
2. Back up the production database.
3. Keep persistent files in a shared location instead of inside a release directory.
4. Prepare the new release fully before switching traffic to it.

5. Use maintenance mode briefly for the migration + symlink switch window.
6. Run migrations with `--force`.
7. Rebuild caches only after the new release has the correct `.env`.
8. Restart queue workers and the websocket process after the code switch.
9. Verify `/ready`, the homepage, login, admin, checkout, queue, websocket/chat, and uploaded-file access.

Recommended approach: use a release directory + `current` symlink. It gives you a practical rollback path for this app, which matters because the codebase contains migrations, long-running queue workers, a websocket server, payment webhooks, and shared storage concerns.

6. Full Step-by-Step Commands

The commands below assume:

- deploy base path: `/var/www/ggwp-boost`
- deploy user: `deploy`
- web group: `www-data`
- branch: `main`
- Nginx + PHP-FPM

Step 1. SSH into the VPS

```
ssh deploy@your-server-ip
```

Connect to the VPS as the deployment user. Use `sudo -i` only when you need elevated permissions for service reloads or initial directory ownership fixes.

Step 2. Define reusable deployment variables

```
APP_NAME="ggwp-boost"
DEPLOY_BASE="/var/www/$APP_NAME"
RELEASES_DIR="$DEPLOY_BASE/releases"
SHARED_DIR="$DEPLOY_BASE/shared"
CURRENT_LINK="$DEPLOY_BASE/current"
BACKUP_DIR="$DEPLOY_BASE/backups"
BRANCH="main"
REPO_URL="git@github.com:your-org/your-repo.git"
RELEASE_NAME="$(date +%Y%m%d%H%M%S)"
RELEASE_DIR="$RELEASES_DIR/$RELEASE_NAME"
```

This keeps the later commands consistent and copy-paste friendly. Replace `REPO_URL` and `BRANCH` with your actual repository and branch.

Step 3. Make sure the deployment directories exist

```
mkdir -p "$RELEASES_DIR" "$SHARED_DIR" "$BACKUP_DIR"
```

Create the standard release, shared, and backup directories if they are not already present.

Step 4. Back up the current production .env

```
cp "$CURRENT_LINK/.env" "$BACKUP_DIR/.env.$(date +%Y%m%d%H%M%S).bak"
```

This gives you a quick recovery copy of the live environment file before you touch anything.

Step 5. Back up the database before the update

```
cd "$CURRENT_LINK"
set -a
. ./env
set +a

MYSQL_PWD="$DB_PASSWORD" mysqldump \
  -h"$DB_HOST" \
  -P"${DB_PORT:-3306}" \
  -u"$DB_USERNAME" \
  --single-transaction \
  --routines \
  --triggers \
  "$DB_DATABASE" \
  > "$BACKUP_DIR/db.$(date +%Y%m%d%H%M%S).sql"

unset MYSQL_PWD
```

This is the safest default for the codebase because production is expected to use MySQL. If your DB is PostgreSQL, use `pg_dump` instead.

Skip only if you already have an external DB backup snapshot taken immediately before deployment. Do not run a migration-based update on this app without a rollback path.

Step 6. Back up persistent uploaded files if they matter to you

```
tar -czf "$BACKUP_DIR/storage-app.$(date +%Y%m%d%H%M%S).tar.gz" -C "$SHARED_DIR" storage/app
```

This is optional but recommended if the site has production uploads such as profile photos, promotion images, or completion proofs.

Step 7. Seed the shared .env and shared storage if this is your first release-based deploy

```
if [ ! -f "$SHARED_DIR/.env" ]; then
  cp "$CURRENT_LINK/.env" "$SHARED_DIR/.env"
fi

if [ ! -d "$SHARED_DIR/storage" ]; then
```

```
mkdir -p "$SHARED_DIR/storage"
rsync -a "$CURRENT_LINK/storage/" "$SHARED_DIR/storage/"
fi
```

Run this once if your existing live site was deployed in-place and you are moving to the safer release-based layout. If your current setup already uses a shared `.env` and shared `storage`, you can skip it.

Step 8. Clone the latest code into a new release directory

```
git clone --branch "$BRANCH" --depth 1 "$REPO_URL" "$RELEASE_DIR"
```

This creates a clean new release directory. It is safer than overwriting the live release in place.

Step 9. Attach shared runtime files to the new release

```
rm -rf "$RELEASE_DIR/storage"
ln -sf "$SHARED_DIR/storage" "$RELEASE_DIR/storage"
ln -sf "$SHARED_DIR/.env" "$RELEASE_DIR/.env"
```

The codebase stores runtime data inside `storage`, including logs, framework state, public uploads, private uploads, and completion proofs. Sharing `storage` across releases prevents upload loss.

Step 10. Ensure the shared storage subdirectories exist

```
mkdir -p \
"$SHARED_DIR/storage/app/public" \
"$SHARED_DIR/storage/app/private" \
"$SHARED_DIR/storage/framework/cache/data" \
"$SHARED_DIR/storage/framework/sessions" \
"$SHARED_DIR/storage/framework/views" \
"$SHARED_DIR/storage/logs"
```

These are the important storage locations for a Laravel app and for this codebase's media handling. The project uses:

- `storage/app/public` via the public disk
- `storage/app/private` via the private/local disks
- `storage/framework/*` for Laravel runtime state

Step 11. Install production PHP dependencies

```
cd "$RELEASE_DIR"
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
```

This installs the production Composer dependencies exactly for the new release. The project requires PHP 8.2 and includes Stripe, Pusher-compatible broadcasting, and the websocket package in Composer dependencies.

Step 12. Install Node dependencies and build production assets

```
cd "$RELEASE_DIR"
npm ci --no-audit --no-fund
npm run build
```

The app uses Vite for frontend builds. Production requires the built asset manifest in `public/build`. Do not run `npm run dev` on the VPS for production.

Step 13. Clear any stale caches in the new release before boot checks

```
cd "$RELEASE_DIR"
php artisan optimize:clear
```

This is important because the app performs startup validation. You do not want stale config cache or route cache from an earlier release leaking into the new boot.

Step 14. Optional but useful: confirm the app can boot in the new release

```
cd "$RELEASE_DIR"
php artisan about
```

This is a quick sanity check that the new release and shared `.env` can boot. If this fails, stop before putting the live app into maintenance mode.

Step 15. Put the live site into Laravel maintenance mode for the cutover window

```
cd "$CURRENT_LINK"
php artisan down --refresh=15 --retry=60
```

Use the standard Laravel maintenance mode for deployment. This is separate from the app's custom admin-controlled maintenance-mode feature backed by system settings.

Do not use the admin UI maintenance toggle for deployment unless you deliberately want the site to remain in application-level maintenance after the release. For deploy cutovers, `artisan down` is simpler and easier to reverse.

Step 16. Run production migrations safely from the new release

```
cd "$RELEASE_DIR"  
php artisan migrate --force
```

This codebase includes many domain tables beyond a basic Laravel app: orders, chat, pending checkouts, payment webhooks, promotions, pages, finance tables, admin workflow tables, and websocket statistics. Run migrations only after a DB backup has been taken.

The repository auto-seeds `PlatformContentSeeder` after `migrate`, `migrate:fresh`, and `migrate:refresh`. That is expected behavior for this project.

Step 17. Point the live symlink at the new release

```
ln -sf "$RELEASE_DIR" "$CURRENT_LINK"
```

This is the actual release switch. From this point onward, your web server and long-running processes should be restarted or reloaded against the new code.

Step 18. Recreate the public storage symlink in the new live release

```
cd "$CURRENT_LINK"  
php artisan storage:link
```

This project's `/ready` endpoint explicitly checks the `public/storage` link, and the app depends on public storage for assets served from the public disk.

Step 19. Build production caches for the new live release

```
cd "$CURRENT_LINK"  
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

These are the safe production cache commands for this codebase. They speed up the app without introducing an unnecessary event-cache dependency into the deployment path.

Step 20. Fix ownership and permissions

```
sudo chown -R deploy:www-data "$DEPLOY_BASE"
sudo find "$SHARED_DIR/storage" -type d -exec chmod 775 {} \;
sudo find "$SHARED_DIR/storage" -type f -exec chmod 664 {} \;
sudo find "$CURRENT_LINK/bootstrap/cache" -type d -exec chmod 775 {} \;
sudo find "$CURRENT_LINK/bootstrap/cache" -type f -exec chmod 664 {} \;
```

The critical writable locations for this project are `storage` and `bootstrap/cache`. Adjust the owner/group if your PHP-FPM or web-server user differs.

Step 21. Restart or reload the runtime services

Queue workers

```
cd "$CURRENT_LINK"
php artisan queue:restart
```

This signals existing `queue:work` workers to restart cleanly and load the new code. It is important because this app queues customer emails and Discord notifications, and defaults those jobs to the `notifications` queue.

PHP-FPM and web server

```
sudo systemctl reload php8.2-fpm
sudo systemctl reload nginx
```

If your PHP-FPM service name differs, change it. If you use Apache instead of Nginx, reload Apache instead.

Websocket service

```
sudo systemctl restart ggwp-websockets
```

Use your actual service name. This app's realtime chat depends on the websocket server staying aligned with the currently deployed code and environment.

Step 22. Bring the site back up

```
cd "$CURRENT_LINK"
php artisan up
```

This removes Laravel maintenance mode. If you had also set the application's own maintenance flag through the admin system, clear that separately in the admin UI or database-backed system settings.

Step 23. Verify the deploy immediately

```
curl -I https://your-domain.example  
curl https://your-domain.example/ready  
tail -n 100 "$CURRENT_LINK/storage/logs/laravel.log"
```

The second command is especially valuable for this repository because `/ready` validates the database, cache, default filesystem writeability, and the `public/storage` symlink.

7. Environment File Setup

The production `.env` should live in the shared directory and be symlinked into each release. Do not commit it to the repository.

Core app values

- `APP_NAME`
- `APP_ENV=production`
- `APP_DEBUG=false`
- `APP_URL=https://your-domain.example`
- `APP_KEY`
- `TRUSTED_PROXIES` if the app sits behind a reverse proxy or load balancer

Database

- `DB_CONNECTION` , `DB_HOST` , `DB_PORT` , `DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`
- The codebase defaults to MySQL in `.env.example`

Session, cache, and queue

- `SESSION_DRIVER` and `SESSION_CONNECTION`
- `CACHE_STORE` , `DB_CACHE_CONNECTION` , `DB_CACHE_LOCK_CONNECTION`
- `QUEUE_CONNECTION` , `DB_QUEUE_CONNECTION` , `DB_QUEUE_BATCHING_CONNECTION` , `DB_QUEUE_FAILED_CONNECTION` , `QUEUE_FAILED_DRIVER`
- This project expects database-backed session/cache/queue state by default, so those backing tables must exist in production

Mail

- `MAIL_MAILER` , `MAIL_HOST` , `MAIL_PORT` , `MAIL_USERNAME` , `MAIL_PASSWORD` , `MAIL_FROM_ADDRESS` , `MAIL_FROM_NAME`
- `TRANSACTIONAL_EMAILS_QUEUE` if you want to override the default queue name for transactional mail
- `CUSTOMER_ORDER_EMAILS_QUEUE` and `CUSTOMER_ORDER_EMAIL_RETRY_FAILED_AFTER_MINUTES`

Broadcasting and websocket config

- `BROADCAST_CONNECTION=pusher` for the default production chat path in this codebase
- `PUSHER_APP_ID` , `PUSHER_APP_KEY` , `PUSHER_APP_SECRET`
- `PUSHER_HOST` , `PUSHER_PORT` , `PUSHER_SCHEME`
- `LARAVEL_WEBSOCKETS_PORT` , `LARAVEL_WEBSOCKETS_PATH` , `WEBSOCKETS_ALLOWED_ORIGINS`

The default local values in `.env.example` use `127.0.0.1:6001` . That is fine for local development, but production browser clients must be able to reach whatever websocket host and scheme you publish. If

your users connect through your domain over HTTPS, make sure the websocket host/scheme reflects that.

Payments

- `STRIPE_ENABLED` , `STRIPE_KEY` , `STRIPE_SECRET` , `STRIPE_WEBHOOK_SECRET`
- `CRYPTOMUS_ENABLED` , `CRYPTOMUS_MERCHANT_ID` , `CRYPTOMUS_API_KEY` , `CRYPTOMUS_BASE_URL` , `CRYPTOMUS_TIMEOUT` , `CRYPTOMUS_INVOICE_LIFETIME`
- `PENDING_CHECKOUT_TTL_HOURS` , `PENDING_CHECKOUT_STALE_RETENTION_HOURS` , `PENDING_CHECKOUT_COMPLETED_RETENTION_HOURS` , `WEBHOOK_PROCESSING_TIMEOUT_MINUTES`

Filesystem and support/brand values

- `FILESYSTEM_DISK=public` unless you intentionally change the default
- `SUPPORT_EMAIL` , `SUPPORT_PHONE` , `COMMUNITY_DISCORD_URL`
- `SOCIAL_*` values and footer legal values

Discord notifications

- `DISCORD_ORDER_WEBHOOK_URL`
- `DISCORD_BOOSTER_APPLICATION_WEBHOOK_URL`
- `DISCORD_CONTACT_WEBHOOK_URL`
- `DISCORD_WITHDRAWAL_WEBHOOK_URL`
- `DISCORD_NOTIFICATIONS_QUEUE` , `DISCORD_PUBLIC_FORM_DEDUPE_WINDOW_MINUTES` , `DISCORD_RETRY_FAILED_AFTER_MINUTES`

Startup validation

- `STARTUP_VALIDATION_ENABLED`
- `STARTUP_VALIDATE_IN_CONSOLE`

In this repository, startup validation is strict in `production` and `staging` . That means a broken payment config, queue config, broadcast config, or mail config can stop the app from booting after deploy. Keep your `production` `.env` stable and always rebuild config cache after changing it.

8. Database / Migration Steps

Production should use MySQL unless you intentionally changed the application to another engine. The codebase includes a local SQLite file and the test suite uses in-memory SQLite, but those are development/testing conveniences, not the primary production target.

1. Take a backup before deployment.
2. Link the production `.env` into the new release.
3. Boot-check the new release with `php artisan about` .
4. Put the live app into maintenance mode.

5. Run `php artisan migrate --force` from the new release.
6. Switch the live symlink only after migrations succeed.
7. Use `php artisan migrate:status` if you want an additional confirmation step.

```
cd "$RELEASE_DIR"  
php artisan migrate --force  
php artisan migrate:status
```

Seeder warning

This project auto-runs `PlatformContentSeeder` after migration commands. That is expected. Do not run demo seeders on production unless you explicitly intend to create demo users or sample data.

9. Frontend Build Steps

The production website uses Vite builds generated from:

- `resources/css/app.css`
- `resources/js/app.js`

The production build outputs hashed files into `public/build`. The Vite dev server is for local development only and should not be part of the production VPS runtime.

```
cd "$RELEASE_DIR"
npm ci --no-audit --no-fund
npm run build
```

Why this matters for this repository

- The layout loads the Vite-built assets for the live site.
- The chat frontend, checkout UI, pricing preview, admin UI helpers, and public social proof scripts all depend on those built files.
- If the build step is skipped, you can end up with missing manifest or stale-asset errors.

10. Storage / Permissions

Storage facts from the codebase

- The default filesystem disk is configurable and `.env.example` uses `public`.
- The codebase also uses a `private` disk and a `local` disk rooted under `storage/app/private`.
- Profile photos and promotion images can be stored on private storage and served through controller routes.
- Completion proofs are stored on the local/private storage path.
- The readiness endpoint checks the `public/storage` link.

Required command

```
cd "$CURRENT_LINK"
php artisan storage:link
```

Recommended ownership pattern

```
sudo chown -R deploy:www-data /var/www/ggwp-boost
sudo chmod -R ug+rwX /var/www/ggwp-boost/shared/storage
sudo chmod -R ug+rwX /var/www/ggwp-boost/current/bootstrap/cache
```

Replace `deploy` and `www-data` with your actual deployment user and web group if they differ.

11. Queue / Scheduler / Realtime Services

Queue workers

This app uses queued customer emails and queued Discord notifications. Production should run a dedicated queue worker continuously.

Recommended queue command

```
php artisan queue:work --queue=notifications,default --sleep=3 --tries=1 --timeout=0
```

Use `php artisan queue:restart` during deploys so workers reload the new release code.

Example systemd unit

```
[Unit]
Description=GGWP Boost Queue Worker
After=network.target

[Service]
User=deploy
Group=www-data
Restart=always
RestartSec=5
WorkingDirectory=/var/www/ggwp-boost/current
ExecStart=/usr/bin/php artisan queue:work --queue=notifications,default --sleep=3 --tries=1 --timeout=0

[Install]
WantedBy=multi-user.target
```

Scheduler

The repository schedules:

- `pending-checkouts:prune` daily
- `discord:retry-dispatches` every ten minutes

Cron entry

```
* * * * * cd /var/www/ggwp-boost/current && php artisan schedule:run >> /dev/null 2>&1
```

Websocket server

Realtime order chat depends on the websocket server. In local development the repo starts it with:

```
php artisan websockets:serve --host=127.0.0.1 --port=6001
```

Example systemd unit

```
[Unit]
Description=GGWP Boost WebSockets Server
```

```
After=network.target

[Service]
User=deploy
Group=www-data
Restart=always
RestartSec=5
WorkingDirectory=/var/www/ggwp-boost/current
ExecStart=/usr/bin/php artisan websockets:serve --host=127.0.0.1 --port=6001

[Install]
WantedBy=multi-user.target
```

If production browser clients connect through a public domain instead of directly to `127.0.0.1:6001`, you must front the websocket server with Nginx or another proxy and set `PUSHER_HOST`, `PUSHER_PORT`, `PUSHER_SCHEME`, and `WEBSOCKETS_ALLOWED_ORIGINS` accordingly.

Supervisor alternative

If you use Supervisor instead of systemd, restart your processes after the deploy:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl restart ggwp-queue:*
sudo supervisorctl restart ggwp-websockets
```

12. Web Server / PHP Runtime Notes

- Your web root must point to the release's `public` directory, typically via the `current` symlink.
- For Nginx, that is usually `/var/www/ggwp-boost/current/public`.
- PHP-FPM must run a version compatible with the codebase: PHP 8.2 or newer.
- After a deployment, reload PHP-FPM and Nginx so the new symlink target and new code are picked up cleanly.

```
sudo systemctl reload php8.2-fpm
sudo systemctl reload nginx
```

If you use Apache, reload Apache instead. If your PHP-FPM service is named differently, adjust the command.

13. Production Optimization Commands

Recommended for this repository:

```
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Why these are appropriate here

- `--optimize-autoloader` improves production Composer autoloading.
- `config:cache` is especially important because this app is heavily environment-driven: payments, queue, mail, broadcast, startup validation, and support metadata all depend on config.
- `route:cache` is safe because the main web routes are controller-based and redirect-based rather than closure-heavy.
- `view:cache` helps because the app is Blade-first.

This guide intentionally does not require `event:cache`. The app already gets most of the value from config, route, and view caching, and keeping the deployment path simpler reduces surprises.

14. Post-Deploy Verification Checklist

Run through this immediately after the release switch and service restarts.

- Homepage loads without a 500 error
- `/ready` returns healthy checks
- Login page loads

- Customer dashboard loads after login
- Admin dashboard loads for an admin account
- Checkout page loads and pricing requests work
- Customer orders page or chat page loads
- Queue worker is processing jobs and not stuck
- Mail test path works in your environment
- Websocket chat connects for an authenticated user
- Profile photos / promotion images / public storage assets are accessible
- Stripe and Cryptomus webhook URLs are still configured correctly after cache rebuild
- No fresh 500 errors appear in `storage/logs/laravel.log`

15. Troubleshooting

Composer install fails

- Check PHP version is at least 8.2
- Check missing PHP extensions
- Run `composer diagnose` if needed

NPM or Vite build fails

- Confirm Node and npm are installed
- Use `npm ci`, not a partial copy of old `node_modules`
- Check for build errors before switching the symlink

Vite manifest errors after deploy

- Run `npm run build` in the new release
- Confirm `public/build/manifest.json` exists in the live release
- Reload PHP-FPM after the new release goes live

App boots with stale config or wrong env values

- Run `php artisan optimize:clear`
- Confirm the live release points to the correct shared `.env`
- Rebuild config cache with `php artisan config:cache`

Permission denied on storage or cache

- Fix ownership for `storage` and `bootstrap/cache`
- Ensure the deploy user and web-server/PHP-FPM user share a writable group

Migrations fail

- Do not switch the symlink

- Inspect the DB error
- Restore the DB backup if a partial migration caused a bad state

Queue not processing jobs

- Check the worker service is running
- Confirm it is listening to `notifications,default`
- Run `php artisan queue:restart` after every deploy

Realtime chat not working

- Confirm websocket service is running
- Check `PUSHER_HOST` , `PUSHER_PORT` , `PUSHER_SCHEME`
- Check browser console and websocket network requests
- Verify allowed origins and reverse-proxy websocket forwarding

Mail not sending

- Check `MAIL_*` values
- Check queue worker health if mail is queued
- Check the mail transport logs or SMTP provider dashboard

419 / CSRF / session issues

- Check `APP_URL` and session domain settings
- Confirm database-backed sessions are working
- Clear and rebuild config cache

403 for admin or role pages

- Check the user role and admin module permissions
- Confirm the account is not suspended

Wrong URL generation after deploy

- Fix `APP_URL`
- Clear config cache and rebuild it
- This matters for reset links, payment success URLs, signed media URLs, and broadcast config

16. Updating in the Future

For future deployments, the repeatable checklist is:

1. SSH in and set deployment variables
2. Back up the database and live `.env`
3. Clone a new release
4. Link shared `.env` and shared `storage`
5. `composer install --no-dev --prefer-dist --optimize-autoloader`
6. `npm ci` and `npm run build`
7. `php artisan optimize:clear`
8. `php artisan down`
9. `php artisan migrate --force`
10. Switch `current` to the new release
11. `php artisan storage:link`
12. `php artisan config:cache`, `route:cache`, `view:cache`
13. `php artisan queue:restart`
14. Reload PHP-FPM / Nginx and restart websockets
15. `php artisan up`
16. Verify `/ready` and the key app paths

17. Rollback / Recovery Notes

If the deploy goes wrong:

1. Put the app in maintenance mode if it is not already down.
2. Point `current` back to the previous release.
3. Reload PHP-FPM and Nginx, restart websockets, and restart the queue worker.
4. If the migration changed the DB in a breaking way, restore the DB backup.
5. Run `php artisan optimize:clear` and rebuild caches for the restored release.
6. Bring the site back up and verify `/ready`.

Example rollback commands

```
PREVIOUS_RELEASE="/var/www/ggwp-boost/releases/previous-timestamp"
ln -sfn "$PREVIOUS_RELEASE" /var/www/ggwp-boost/current

cd /var/www/ggwp-boost/current
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

```
php artisan queue:restart

sudo systemctl reload php8.2-fpm
sudo systemctl reload nginx
sudo systemctl restart ggwp-websockets

php artisan up
```

Code rollback is easy with the symlink strategy. Database rollback is not. That is why the database backup step is mandatory for safe production updates on this repository.

18. Final Command Summary

This is the condensed command-only version for future reuse.

```
ssh deploy@your-server-ip

APP_NAME="ggwp-boost"
DEPLOY_BASE="/var/www/$APP_NAME"
RELEASES_DIR="$DEPLOY_BASE/releases"
SHARED_DIR="$DEPLOY_BASE/shared"
CURRENT_LINK="$DEPLOY_BASE/current"
BACKUP_DIR="$DEPLOY_BASE/backups"
BRANCH="main"
REPO_URL="git@github.com:your-org/your-repo.git"
RELEASE_NAME="$(date +%Y%m%d%H%M%S)"
RELEASE_DIR="$RELEASES_DIR/$RELEASE_NAME"

mkdir -p "$RELEASES_DIR" "$SHARED_DIR" "$BACKUP_DIR"
cp "$CURRENT_LINK/.env" "$BACKUP_DIR/.env.$(date +%Y%m%d%H%M%S).bak"

cd "$CURRENT_LINK"
set -a
. ./env
set +a
MYSQL_PWD="$DB_PASSWORD" mysqldump -h"$DB_HOST" -P"${DB_PORT:-3306}" -u"$DB_USERNAME" --single-transaction --routines --triggers "$DB_DATABASE" > "$BACKUP_DIR/db.$(date +%Y%m%d%H%M%S).sql"
unset MYSQL_PWD

if [ ! -f "$SHARED_DIR/.env" ]; then cp "$CURRENT_LINK/.env" "$SHARED_DIR/.env"; fi
if [ ! -d "$SHARED_DIR/storage" ]; then mkdir -p "$SHARED_DIR/storage" && rsync -a "$CURRENT_LINK/storage/" "$SHARED_DIR/storage/"; fi

git clone --branch "$BRANCH" --depth 1 "$REPO_URL" "$RELEASE_DIR"
rm -rf "$RELEASE_DIR/storage"
ln -sfn "$SHARED_DIR/storage" "$RELEASE_DIR/storage"
ln -sfn "$SHARED_DIR/.env" "$RELEASE_DIR/.env"

mkdir -p "$SHARED_DIR/storage/app/public" "$SHARED_DIR/storage/app/private"
"$SHARED_DIR/storage/framework/cache/data" "$SHARED_DIR/storage/framework/sessions"
"$SHARED_DIR/storage/framework/views" "$SHARED_DIR/storage/logs"

cd "$RELEASE_DIR"
```

```
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
npm ci --no-audit --no-fund
npm run build
php artisan optimize:clear
php artisan about

cd "$CURRENT_LINK"
php artisan down --refresh=15 --retry=60

cd "$RELEASE_DIR"
php artisan migrate --force

ln -sfn "$RELEASE_DIR" "$CURRENT_LINK"

cd "$CURRENT_LINK"
php artisan storage:link
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan queue:restart

sudo chown -R deploy:www-data "$DEPLOY_BASE"
sudo find "$SHARED_DIR/storage" -type d -exec chmod 775 {} \;
sudo find "$SHARED_DIR/storage" -type f -exec chmod 664 {} \;
sudo find "$CURRENT_LINK/bootstrap/cache" -type d -exec chmod 775 {} \;
sudo find "$CURRENT_LINK/bootstrap/cache" -type f -exec chmod 664 {} \;

sudo systemctl reload php8.2-fpm
sudo systemctl reload nginx
sudo systemctl restart ggwp-websockets

php artisan up

curl -I https://your-domain.example
curl https://your-domain.example/ready
tail -n 100 "$CURRENT_LINK/storage/logs/laravel.log"
```